SmartBudget

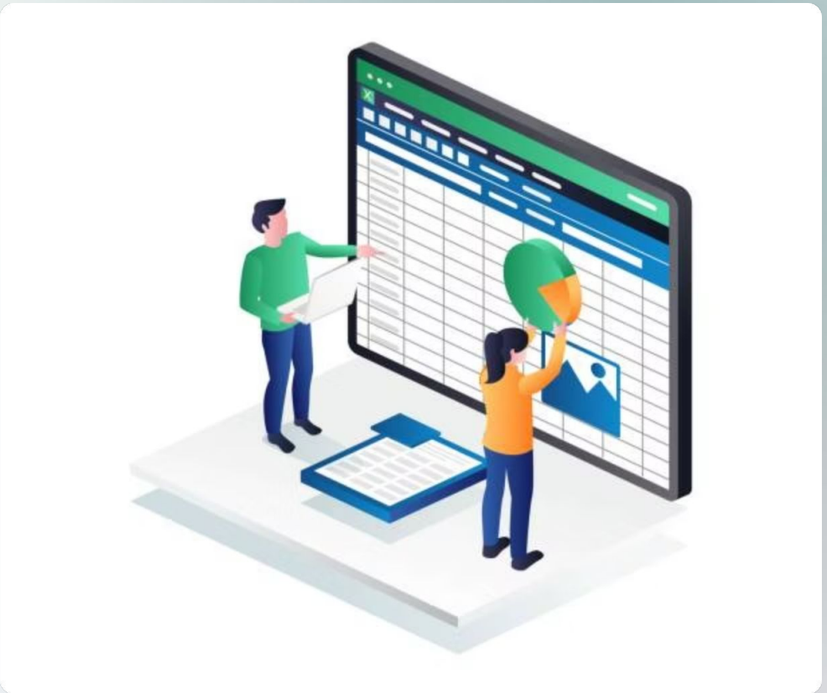
Personal Finance Tracker

SmartBudget

Personal Finance Tracker

Final Project Presentation • DIGT2107 – Practice of Software Development • Team #3

Alia Hagi-Dhaffe, Mohsen Pasdar, Muhammed Ahmed



Why SmartBudget?

Manual Spreadsheets

Time-consuming and inconsistent tracking

Overly Complex Apps

Many apps lack privacy focus

Offline Solution

Simple desktop budgeting tool

Project Goal & Users

Project Goal

- Simple desktop finance tracker
- Offline-first design
- Easy to use interface
- Clear financial overview

Target Users

- Students
- Young professionals
- Privacy-focused users
- Anyone wanting simple budgeting support

Main Objectives

01

Record Transactions

Income and expenses tracking

02

Manage Data

Edit, delete, and filter transactions

03

Set Budgets

Monthly limits with alerts

04

Generate Reports

Summaries and spending charts

05

Persistent Data

Save data across sessions

Development Journey

1 **Phase 1**

Vision, planning, requirements, use cases

2 **Phase 2**

Testing plan, prototype, mid-project report

3 **Phase 3**

Design, GUI, implementation, testing, documentation

Agile / Iterative Approach

How We Built It

Implementation

Built incrementally

Requirements

Guided design decisions

Testing

Validation and refinement

Design

Guided implementation

Incremental development with each deliverable built on the previous one. Requirements guided design, design guided implementation, and testing supported validation and refinement.

Core Requirements

Functional

Non-Functional

Functional Requirements

- FR1 Transactions
- FR2 Browsing and filters
- FR3 Budgets and alerts
- FR4 Summaries and charts

Non-Functional Requirements

- Usability
- Performance
- Reliability
- Privacy / offline use
- Maintainability

Key Use Cases

The primary workflows users will engage with in SmartBudget:

Record Transactions

Quickly log income and expenses.

Edit / Delete

Modify or remove entries with ease.

View / Filter

Browse and filter transactions by category or date.

Set Monthly Budget

Define spending limits for various categories.

Overspending Alert

Get notified when nearing or exceeding budget limits.

Summary Report

Generate detailed financial summaries.

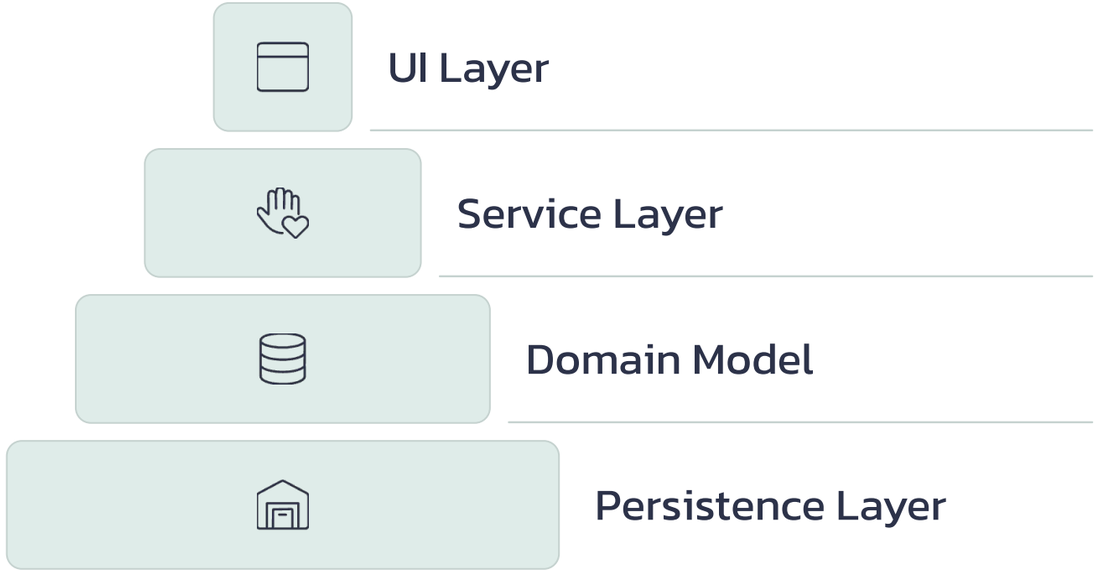
Spending Chart

Visualize spending patterns with intuitive charts.

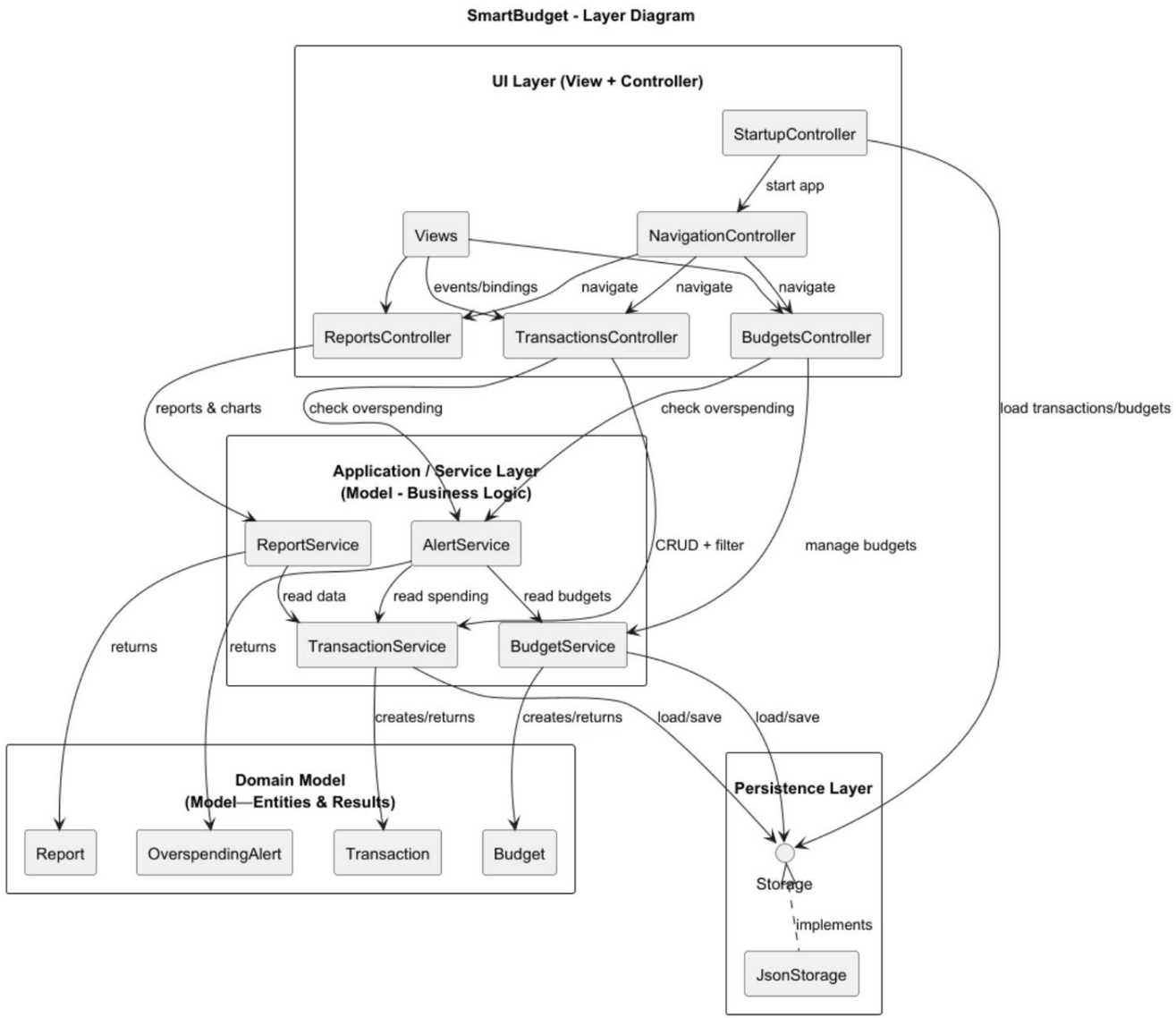
Load Saved Data

Automatically load previous data on startup.

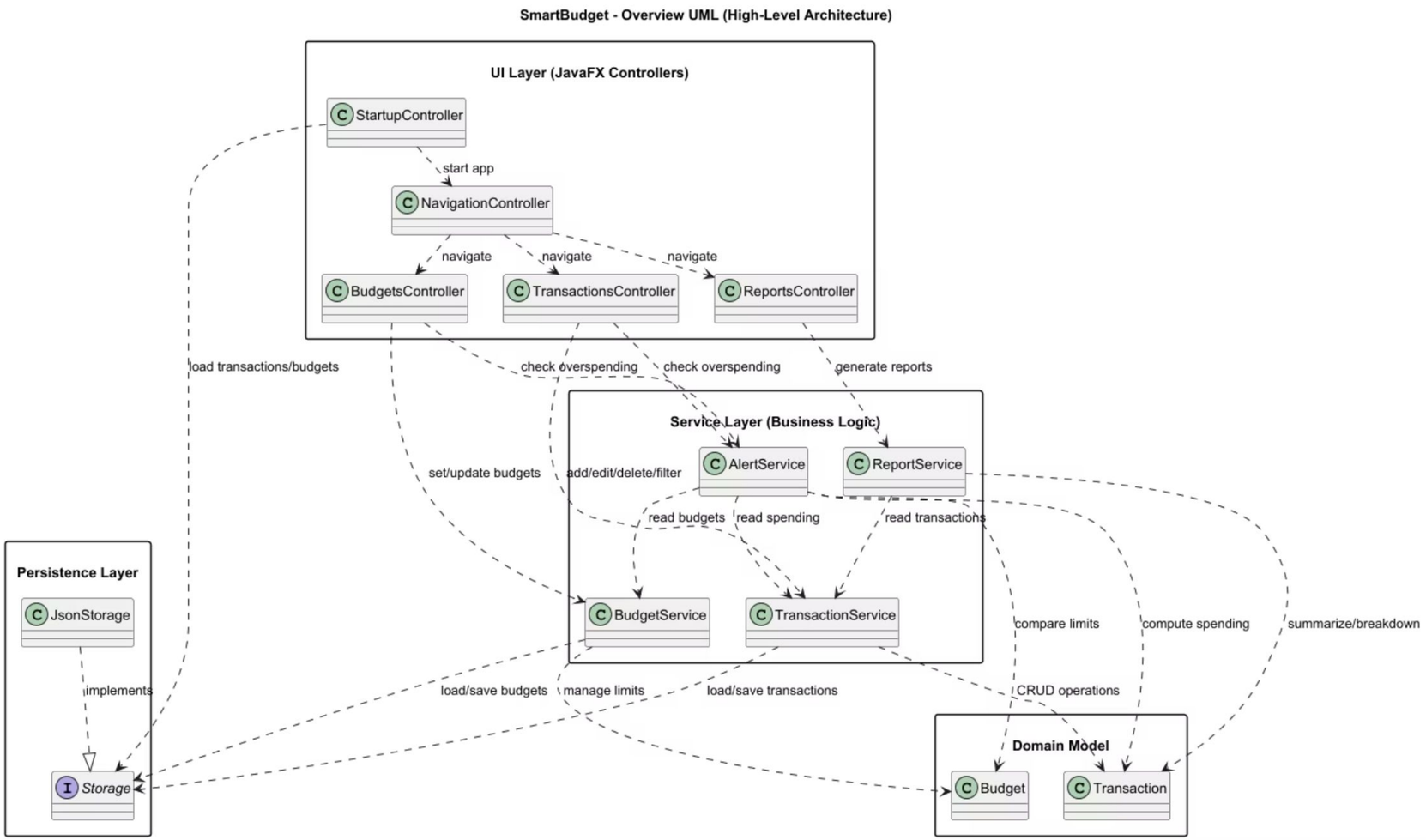
System Architecture



Clear separation makes testing, maintenance, and extension easier.



Main System Modules



Transactions

- Add, edit, delete, filter
- Comprehensive record keeping

Budgets + Alerts

- Set custom spending limits
- Detect overspending in real-time

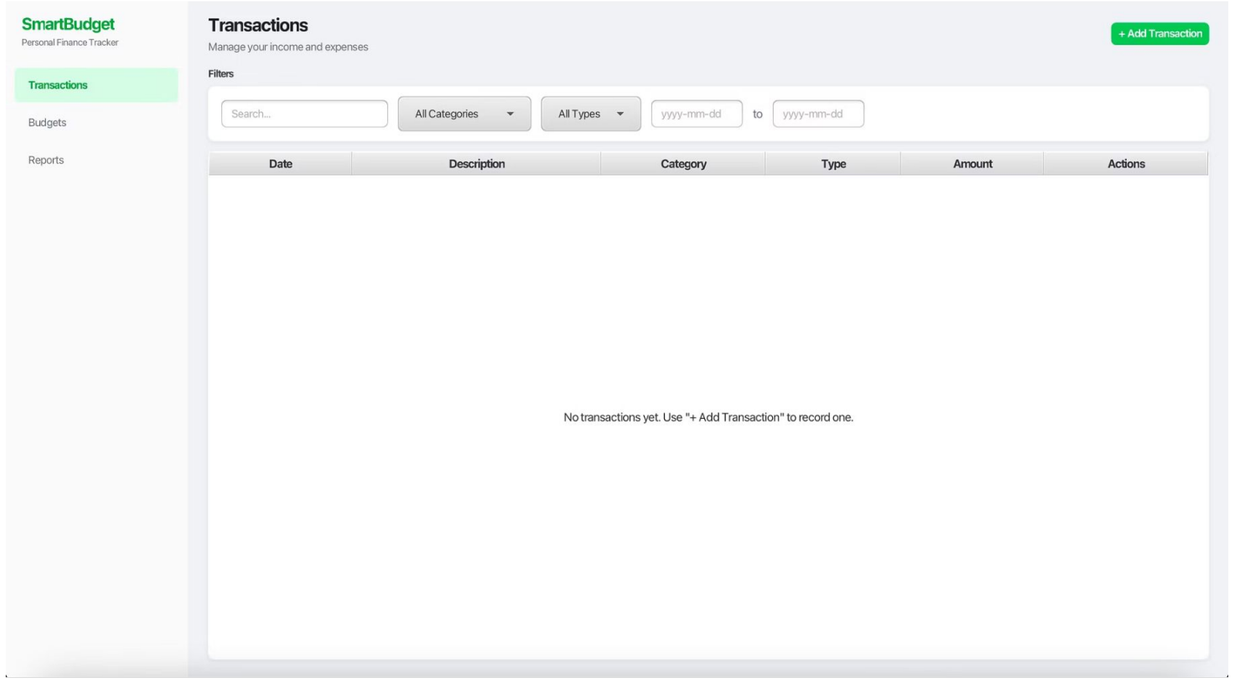
Reports

- Generate financial summaries
- View total spending & category breakdowns

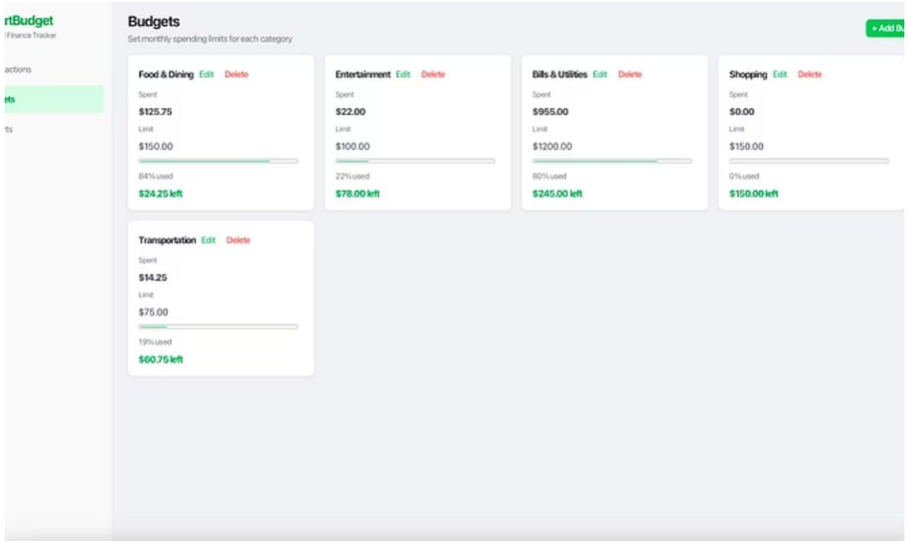
User Interface & Navigation

Transactions | **Budgets** | **Reports**

Users can switch between these main sections, while interactive forms, confirmation modals, and alerts ensure a clear and guided interaction throughout the application.



Key Features

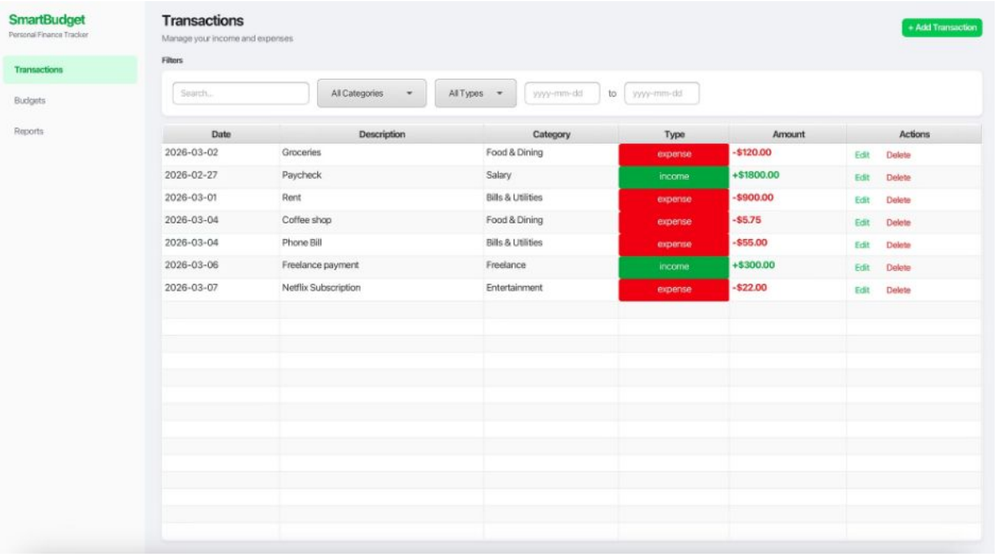


Transactions

Add, edit, delete, and filter transactions by criteria

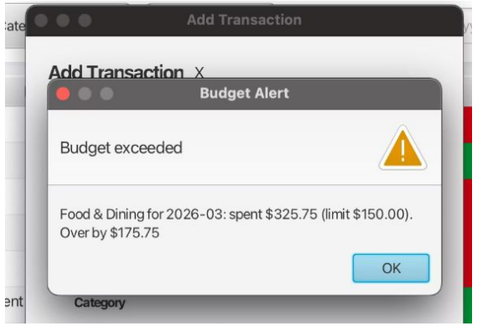
Reports

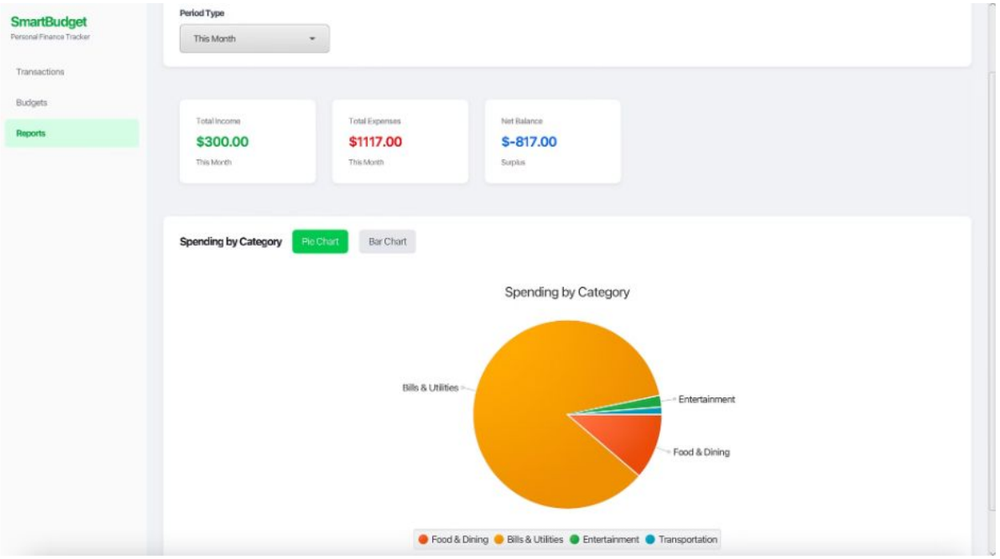
Generate summaries, totals, and category breakdowns with charts



Budgets & Alerts

Set monthly limits and receive overspending notifications





Demo

Google Docs

SmartBudgetDemo.mov

Testing Approach

Traceability
Ensuring every requirement is linked to specific test cases.

Detailed Cases
Thorough test scenarios covering expected behavior and edge cases.

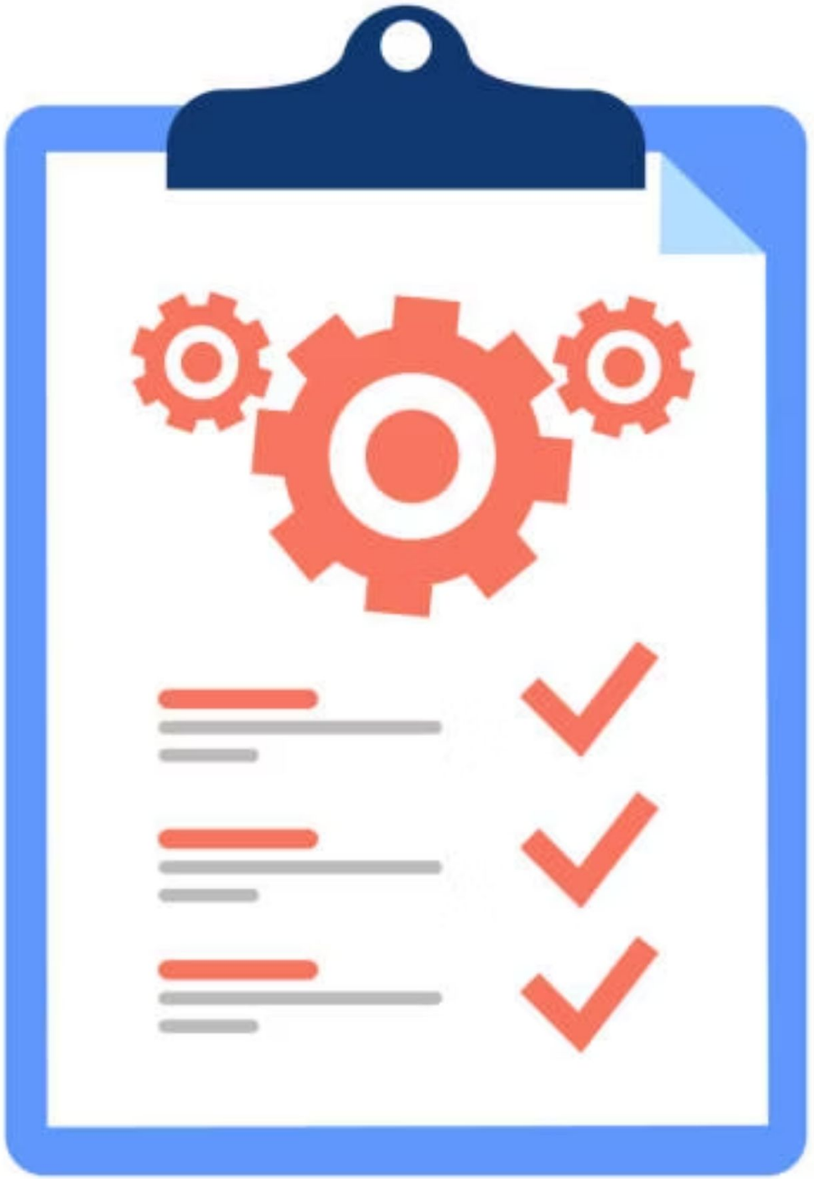
Unit Testing
Validating individual components in isolation for correctness.

Functional Testing
Verifying that each feature performs according to specifications.

Integration Testing
Checking seamless interaction between different system modules.

Persistence Testing
Confirming reliable data saving and loading across sessions.

Manual GUI
Human verification of the user interface for usability and experience.



Testing Results

Our testing approach led to strong and reliable outcomes:

Quality Outcomes

Core workflows validated: All critical user journeys confirmed as fully functional.

End-to-end system behavior checked: Seamless interaction across all modules ensured.

Save/load behavior tested: Data persistence confirmed reliable across sessions.

Reports and budgets integrated: Features correctly display and apply financial rules.

Final system ready for demo and submission

Challenges

Navigating the development process of SmartBudget presented several key challenges:

Modular Design

Ensuring a clean, scalable architecture with distinct, independent components.

Persistence Integration

Implementing reliable data saving and loading.

Cross-Module Consistency

Maintaining a consistent user experience and data handling across modules.

Expanding Test Coverage

Developing comprehensive tests for all functionalities and edge cases.

Usability & Validation

Refining the usability and ensuring validation is strong.

Solutions

How We Solved Them

Layered Architecture

Service-Based Logic

Cleaner Separation of Concerns

Incremental Refactoring

More Complete Testing

Stronger Validation

Persistence + Startup Loading

Our targeted solutions directly addressed the development challenges, resulting in a more robust and user-friendly SmartBudget application.

Future Enhancements

Possible Next Steps

1. Recurring Transactions
2. Export Functionality
3. Enhanced Dashboard Analytics
4. Multi-User Support
5. Cloud Synchronization
6. Smarter Financial Insights

Our vision for SmartBudget extends to a series of impactful future enhancements, focusing on greater convenience, deeper insights, and collaborative functionality to provide an even more comprehensive financial management tool.

What We Achieved

- Full project life cycle completed
- Working desktop application
- Core requirements implemented
- Design aligned with implementation
- Testing used for validation

SmartBudget successfully transformed an initial idea into a complete, functional personal finance tool, ready to empower users with better financial management.

Thank you for listening!

Any questions?